

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

PRACTICAL 7

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 08-08-2026	Date/Time of Submission: 08-08-2026

6. Implement Decision Tree regression on a regression dataset.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
data = pd.read_csv("iris - iris - iris - iris.csv")
X = data[['sepal_length']]
y = data['petal_length']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
model = DecisionTreeRegressor()
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
new_data = pd.DataFrame([[6.0]], columns=['sepal_length'])
prediction = model.predict(new_data)
print("Predicted Petal Length for Sepal Length 6.0:", prediction[0])
```

Output:-

```
Training Accuracy: 86.44982464055326
Testing Accuracy: 72.01267791587007
Predicted Petal Length for Sepal Length 6.0: 4.6
Ritesh Chaubey
```

7. Implement k-Nearest Neighbors (kNN) classifier.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
data = pd.read_csv("iris - iris - iris - iris.csv")
```

```
X = data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = data['species']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
new_data = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
)
prediction = model.predict(new_data)
print("Predicted Species:", prediction[0])
```

Output:-

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Species: setosa
Ritesh Chaubey
```

8. Implement multiclass classification using Logistic Regression or Decision Tree.

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
data = pd.read_csv("iris - iris - iris.csv")
X = data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = data['species']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
```

```
new_data = pd.DataFrame(  
    [[5.1, 3.5, 1.4, 0.2]],  
    columns=['sepal_length', 'sepal_width', 'petal_length', 'petal_width']  
)  
prediction = model.predict(new_data)  
print("Predicted Species:", prediction[0])
```

Output:-

```
Training Accuracy: 100.0  
Testing Accuracy: 95.55555555555556  
Predicted Species: setosa  
Ritesh Chaubey
```

(Perform Logistic Regression, Decision Tree, and kNN on the same dataset)

Output:-**1.Code:-**

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeRegressor  
data = pd.read_csv("housing.csv")  
X = data[['median_income']]  
y = data['median_house_value']  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.3,  
    random_state=1  
)  
model = DecisionTreeRegressor()  
model.fit(X_train, y_train)  
print("Training Accuracy:", model.score(X_train, y_train)*100)  
print("Testing Accuracy:", model.score(X_test, y_test)*100)  
new_data = pd.DataFrame([[5.0]], columns=['median_income'])  
prediction = model.predict(new_data)  
print("Predicted House Price:", prediction[0])  
print("Ritesh Chaubey ")
```

Output:-

```
Training Accuracy: 83.85509851405484  
Testing Accuracy: 13.233356697501698  
Predicted House Price: 290023.1538461539  
Ritesh Chaubey
```

2.Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
data = pd.read_csv("housing.csv")
data['price_category'] = data['median_house_value'].apply(
    lambda x: 'High' if x >= 200000 else 'Low'
)
X = data[['median_income']]
y = data['price_category']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
new_data = pd.DataFrame([[5.0]], columns=['median_income'])
prediction = model.predict(new_data)
print("Predicted Price Category:", prediction[0])
print("Ritesh Chaubey ")
```

Output:-

```
Training Accuracy: 82.19130675526024
Testing Accuracy: 71.46317829457365
Predicted Price Category: High
Ritesh Chaubey
```

3.Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
data = pd.read_csv("housing.csv")
data['price_category'] = data['median_house_value'].apply(
    lambda x: 'High' if x >= 200000 else 'Low'
)
X = data[['median_income']]
y = data['price_category']
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
y,  
test_size=0.3,  
random_state=1  
)  
model = DecisionTreeClassifier()  
model.fit(X_train, y_train)  
print("Training Accuracy:", model.score(X_train, y_train)*100)  
print("Testing Accuracy:", model.score(X_test, y_test)*100)  
new_data = pd.DataFrame([[5.0]], columns=['median_income'])  
prediction = model.predict(new_data)  
print("Predicted Price Category:", prediction[0])  
print("Ritesh Chaubey ")
```

Output:-

```
Training Accuracy: 90.37929125138427  
Testing Accuracy: 69.23449612403101  
Predicted Price Category: High  
Ritesh Chaubey
```